TECHNICAL DEEP-DIVE

Inaya Ecosystem — Developer Reference

Contract functions, API routes, data models, and SDK signatures — a lookup reference grounded directly in the real code.

Document INAYA-DEV-2026-V1 · Classification Internal · August 2026

SECTION 01

Repo & Entry-Point Map

PATH	ENTRY POINT
contracts/ (repo root)	*.sol — Hardhat project, hardhat.config.js at repo root
scripts/ (repo root)	deploy*.cjs / deploy*.js, verify-*.cjs, simulate-security-nodes.js
inaya-network-dapp/	Next.js 14 App Router — src/app/, src/lib/, src/app/api/
inaya-network-dapp/custody-sdk/	src/index.js (InayaKernel), src/crypto.js, src/contracts.js
inaya-network-dapp/custody-sdk/packages/node-daemon/	bin/inaya-node-daemon.js (commander CLI)
inaya-mobile/	Expo — App.js, src/screens/, src/utils/, src/providers/
inaya-desktop/	src-tauri/src/lib.rs, tauri.conf.json
inaya-dapp-desktop/	src-tauri/src/lib.rs, tauri.conf.json

SECTION 02

Contract Function Reference

InayaStaking.sol

Constructor(stakingToken, rewardToken) — both = $INAYA address.

- stake(uint256 amount, uint256 lockPeriodDays) — lockPeriodDays ∈ {0,30,90}, multiplier 1.00x/1.25x/1.50x, locked in on first stake
- withdraw(uint256 amount) — reverts if block.timestamp < lockExpiry
- claimReward() / exit() (withdraw all + claim)
- earned(address) / rewardPerToken() / getUserTier() → "None"/"Standard"/"Enterprise Priority"
- owner-only: setRewardRate, fundRewardPool, setEnterpriseTierThreshold, recoverForeignToken (blocked on staking/reward token)
- events: Staked, Withdrawn, RewardPaid, RewardRateUpdated, RewardPoolFunded

InayaEgressTimelockVault.sol

Constructor(inayaToken, usdtToken, stakingContract, operationalTreasury).

- executeSemiAnnualHarvest() — public, callable by anyone once currentEpochStart + 180 days has passed; sweeps full INAYA balance to stakingContract via raw transfer (not fundRewardPool)
- forwardUSDT() — public, sweeps full USDT balance to operationalTreasury
- getNextHarvestCountdown() view
- owner-only: setStakingContract, setOperationalTreasury

InayaCorporateEscrow.sol

Constructor(usdtToken) only.

- createEscrow(address corporate, address node, uint256 totalAmount) — safeTransferFrom(caller) into escrow, MONTHS=12 fixed schedule
- releaseMonthlyPayout(uint256 scheduleId) — public, callable by anyone once due
- owner-only: setMonthSeconds (testnet-only timing override), reassignNode
- events: EscrowCreated, MonthlyReleased, EscrowClosed

InayaNodeRegistry.sol

Constructor(usdtToken, verifierWallet). SCOPE NOTE in header: metrics are coordinator-verified, NOT cryptographic proof-of-storage.

- registerNode(uint256 capacityGB) — self-serve
- updateNodeMetrics(...) — verifier-only
- queueSettlement / queueSettlementsBatch — verifier-only, computes commission (30/40/50% by Entry/Mid/Enterprise tier), does not move funds
- releaseSettlement / releaseSettlementsBatch — public, callable by anyone, only after SETTLEMENT_DELAY = 36 hours
- owner-only: setVerifierWallet, setTierThresholds, withdrawExcessReserve

InayaProofRegistry.sol

Constructor(custodyAddress) — immutable IInayaCustody reference.

- registerMerkleRoot(bytes32 fileHash, bytes32 merkleRoot, uint256 chunkCount, address node) — requires msg.sender == custody.assets(fileHash).owner; node must be pre-approved via setNodeRegistered
- verifyChunkProof(...) — onlyOwner today (backend-checked); header comment documents a planned permissionless + slashing path
- events: MerkleRootRegistered, ProofVerified, NodeRegistrationChanged

InayaCustody (interface only — IInayaCustody)

Deployed, source not tracked in this repo. Interface as referenced from InayaProofRegistry:

- assets(bytes32 fileHash) view → {owner, cidAlpha, cidBeta, size, timestamp}
- batchRegisterAssets(bytes32[] fileHashes, uint256[] fileSizes, string[] shardACIDs, string[] shardBCIDs)
- event AssetRegistered

RevenueRouter (interface only)

Deployed, source not tracked in this repo. Called via inline ABI from page.js and the Stripe webhook route:

- processCorporateInvoice(uint256 usdtAmount) external

Security Layer contracts

InayaThreatRegistry (status ledger, updated only by Reporter), InayaThreatReporter (confirmThreat(bytes32 threatId, uint8 category, uint16 confidenceBps, bytes32 contributingNodesHash) — relayer-only), InayaNodeReputation (checkpointReputation(...) — relayer-only, periodic), InayaSecurityPolicy (publishPolicy(uint256 version, bytes32 policyHash, string policyURI) — owner/relayer-only).

Oracle & Automation Layer contracts (deployed to BSC Testnet)

InayaOracleRegistry (registerSource(bytes32 id, string dataType, address submitter, uint256 updateFrequency) — owner-only; isAuthorizedSubmitter(id, address) view), InayaOracleAdapter (submitData(bytes32 id, uint256 value, uint256 reportedTimestamp) — reverts on future/stale timestamp, sub-minimum-interval, or excess deviation; getLatestData(id) / isStale(id) views), InayaAutomationRegistry (registerTask(bytes32 id, address target, bytes4 selector, string condition) — owner-only; recordExecution(id, bool success, uint256 nextEligible, bytes32 txHash) — worker/owner-only, pure audit-trail write, never forwards a call to target).

SECTION 03

custody-sdk (InayaKernel) API Reference

FUNCTION	PURPOSE
generateSecureSalt()	16-byte random salt via crypto.getRandomValues
deriveVaultKey({passkey, salt, iterations=100000, algo})	PBKDF2 → 32-byte AES key
disperseAndSlice({file, encryptionKey})	Encrypt (AES-GCM-256) + base64-midpoint-split into shardAlpha/shardBeta
anchorToLedger(...)	Calls InayaCustody.batchRegisterAssets() with CIDs (not raw shard bytes)
retrieveAndReconstruct({connection, fileHash, passkey, fetchShard})	Reads assets(fileHash), fetches both shards, reassembles
reconstructAndDecrypt({shardAlpha, shardBeta, passkey})	Concat → split salt/IV/ciphertext → re-derive key → AES-GCM decrypt
encryptForPublicKey / decryptWithSecretKey / deriveEncryptionKeypairFromSignature	X25519 + HKDF-SHA256 + XChaCha20-Poly1305 sealed-box — re-wraps the passkey for sharing, not the file
connectWallet, approveFeeTokens	ethers v6 wallet/allowance helpers
Staking.{stake,unstake,claimReward,calculateReward,getStakedBalance}	Thin wrapper over InayaStaking
Payments, Metadata, Analytics, events (EventEmitter)	Supporting modules
errors.{InayaError,InayaValidationError,InayaWalletError,InayaContractError,InayaNetworkError}	Typed error classes

> Crypto primitives are @noble/hashes, @noble/ciphers, @noble/curves (pure JS) rather than crypto.subtle — chosen specifically for React Native compatibility, per an inline comment in crypto.js.

SECTION 04

node-daemon CLI Reference

COMMAND	EFFECT
login	Reads INAYA_PRIVATE_KEY (env or prompt), encrypts at rest (PBKDF2-SHA256 200k iter + AES-GCM-256), writes ~/.inaya/node-daemon/config.json
register <capacityGB>	InayaNodeRegistry.registerNode(capacityGB) on-chain + POST /api/nodes/register off-chain
start	Foreground 5-min heartbeat loop → POST /api/nodes/heartbeat with {nodeId, operatorWallet, totalCapacityGB, usedCapacityGB:0, shardsStored:0} (last two hardcoded 0 — no shard-storage capability exists)
report <indicator> --category <cat>	Security Layer signed observation → POST /api/security/report
service install / uninstall	Windows background service via node-windows

> package.json description states it plainly: registers your node on-chain and reports heartbeat/telemetry. Does not store or serve shards. NODE_REGISTRY_ABI in this package has exactly two functions (registerNode, nodes) — no stake/settlement/withdraw calls exist in the daemon's on-chain surface.

SECTION 05

Backend lib/ Reference

Every file below follows the same shape: getXCollections(), ensureXIndexes() (module-level indexesEnsured guard), validateXInput() (throws, fail-closed).

FILE	DOMAIN
security.js	Threat reports/threats/policy/events/reputation-cache, computeThreatConfidence, getPublicSecurityStats
ai-security-tools.js	Security Assistant tool declarations + dispatcher
learn.js / learnConfig.js / youtube.js	Learn saved/progress/search-cache/video-cache, category+collection config, cached YouTube Data API client
ai-learn-tools.js	Learn Tutor tool declarations + dispatcher
orgs.js / orgPlans.js	Business Workspace auth (sessions, magic-links), org/plan definitions
document-workflow.js / document-permissions.js	Document state machine + activity log; access resolution + share tokens
ai-business-tools.js	Business Assistant tool declarations + dispatcher (permission-aware)
dataroom.js	Investor Data Room visitors/sessions/documents/views
activity.js	DAU/WAU ping recording + stats
watcherPioneer.js	Watcher Pioneer enrollment/qualification
feedback.js	Bug reports / idea submissions
metadata-auth.js	Shared signed-message verification helper (verifyMetadataAuth)
email.js	Resend-backed transactional email (magic links, Data Room verify)

SECTION 06

API Route Reference (by domain)

DOMAIN	ROUTES
Security (public)	GET /api/security/{threat,feed,stats,policy}, POST /api/security/report, GET /api/security/reputation/[address], POST /api/security/events, GET /api/security/events
Security (admin)	GET /api/admin/security/{threats,nodes}, POST /api/admin/security/threats/[id]/override, GET|POST /api/admin/security/policy
Security (AI)	POST /api/ai/security-chat
Security (cron)	GET /api/security/cron/checkpoint-reputation (CRON_SECRET-gated)
Learn	GET /api/learn/config, GET /api/learn/search, GET /api/learn/video/[videoId], GET|POST /api/learn/saved, DELETE /api/learn/saved/[videoId], GET|POST /api/learn/progress, POST /api/learn/report, POST /api/learn/analytics
Learn (AI)	POST /api/ai/learn-chat
Business Workspace (auth)	POST /api/orgs/create, /api/orgs/login/google, magic-link login/verify
Business Workspace (org)	department/project CRUD, document upload/list/retrieve/transition, permissions grant/update/revoke, share create/list/revoke, GET /api/orgs/share/[token] (public resolve)
Business Workspace (billing)	GET /api/orgs/billing, POST /api/orgs/billing/checkout, POST /api/orgs/billing/portal, GET /api/orgs/billing/plans
Business Workspace (AI)	POST /api/ai/business-chat
Data Room (public)	POST /api/dataroom/request-access, GET /api/dataroom/verify, GET /api/dataroom/session, POST /api/dataroom/accept-nda, GET /api/dataroom/documents, GET /api/dataroom/documents/[id]/stream, POST /api/dataroom/documents/[id]/view-event
Data Room (admin)	GET|POST /api/admin/dataroom/documents, DELETE /api/admin/dataroom/documents/[id], GET /api/admin/dataroom/visitors, POST /api/admin/dataroom/visitors/[id]/revoke
Nodes	POST /api/nodes/register, POST /api/nodes/heartbeat, GET /api/nodes/settlements/release (cron)
Referrals	POST /api/referrals/{activate,initiate,webhook}, GET /api/referrals/{status,history,leaderboard}
Watcher Pioneer	POST /api/watcher/{enroll,qualify}, GET /api/watcher/status

DOMAIN	ROUTES
Activity	POST /api/activity/ping, GET /api/admin/activity
Other	POST /api/faucet, POST /api/stripe-webhook, POST /api/create-payg-checkout-session, POST /api/feedback/submit, POST /api/feedback/upload, GET /api/admin/feedback

SECTION 07

MongoDB Collections Reference

COLLECTION GROUP	KEY COLLECTIONS
Security	security_reports, security_threats, security_policy, security_events, security_reputation_cache
Learn	learn_saved, learn_progress, learn_search_cache (TTL 24h), learn_video_cache (TTL 24h), learn_reports, learn_analytics_events
Business Workspace	orgs, org_members, sessions, magic_links, departments, projects, documents, document_permissions, document_shares, document_activity, project_members
Data Room	dataroom_documents, dataroom_visitors, dataroom_magic_links, dataroom_sessions, dataroom_views
Activity	activity_pings — one doc per {surface, identityId, date}, unique compound index
Referrals / Watcher Pioneer / Feedback	referrals, watcher_enrollments, feedback_submissions (each with their own lib.js as listed in Section 05)

SECTION 08

AI Assistant Tool-Calling Pattern

Identical shape across all three assistants (src/app/api/ai/{business,security,learn}-chat/route.js):

- Model: gemini-flash-latest, thinkingConfig: { thinkingLevel: "low" }, maxOutputTokens: 800
- MAX_TOOL_ROUNDS = 5 — a bounded loop of generateContent → check functionCalls → run tools → feed functionResponse back in
- generateContentWithRetry() — retries on HTTP 429/503 with a [700ms, 1800ms] backoff schedule
- Route-level: maxDuration = 90, not streaming (deliberate — unpredictable number of tool round-trips before a final answer)
- Per-assistant: buildXContext() computed once per request, X_TOOL_DECLARATIONS (Gemini Type.OBJECT schemas), runXTool(name, args, ctx) dispatcher, xSystemInstruction() builder

SECTION 09

Auth Patterns — Technical Detail

PATTERN	IMPLEMENTATION DETAIL
Wallet signed-message	ethers.verifyMessage(reconstructedMessage, signature) === expectedAddress, 5-minute timestamp freshness window, reimplemented per-feature (metadata-auth.js has the canonical version, security.js/watcherPioneer.js reimplement locally by convention)
Magic-link	generateToken() (random), hashToken() (stored hashed, never plaintext), TTL-indexed Mongo collection, consumeLoginToken() atomic findOneAndUpdate to prevent replay
Session (web)	HttpOnly cookie, createSession(email) helper in orgs.js
Session (mobile)	Bearer token, same session concept, stored via expo-secure-store
Share tokens	generateShareToken()/hashShareToken(), atomic findOneAndUpdate-based single-consume pattern (document-permissions.js), reused as a template for Data Room's magic-link tokens

SECTION 10

Testing & Verification Conventions

- Hardhat tests: test/security-layer.test.js (17 tests — access control, confirm/re-confirm, false-positive override, 4-signer simulation)
- Node backend tests: test/*.test.mjs run via node --test against a real MongoDB instance (not mocked) — security.test.mjs (11), learn.test.mjs, dataroom.test.mjs, activity.test.mjs, feedback.test.mjs, watcher-pioneer.test.mjs — 140/140 passing as of the Security Layer build, no regressions
- On-chain simulation: scripts/simulate-security-nodes.js — 4 throwaway signers report a clearly-fake test indicator end-to-end through to a real on-chain confirmThreat() transaction on BSC Testnet
- Manual browser verification is the norm for UI work in this codebase — curl/fetch checks for API correctness, then a real browser pass (or explicit statement that one hasn't happened yet) before anything is called "verified"